

Laxmi Charitable Trust's
Sheth L.U.J College of Arts & Sir M.V. College of Science and Commerce
Department of Information Technology (B.Sc.I.T Semester V)

Full Stack Development Using Mern Stack

Module - II

Practical – VIII
(Mini Project)

Roll No.:T054	Name:Priti Yadav
Class:TYIT	Batch:2
Date of Assignment: 23/09/26	Date/Time of Submission: 23/09/26

Project Title: College Canteen Pre-Order System

Develop a web-based College Canteen Pre-Order System. It should allow students to:

1. View the available food items and their prices.
2. Enter student details such as name, roll number, and course.
3. Select a food item and quantity.
4. Place an order using a POST API.
5. Store the order details in MongoDB.
6. Retrieve the stored orders using a GET API.
7. Display the available food items and placed orders on the React page.

APIs to be implemented

- GET /food – Retrieve food items.
- POST /orders – Add a new food order.
- GET /orders – Retrieve placed orders.

Technologies

- React
- Axios
- Node.js
- Express.js
- MongoDB

Expected Output

The final application should contain:

- Food Item List
- Student Order Form
- Place Order button
- Order List/Table

Design the interface and decide the appropriate fields, MongoDB document structure, and API flow.

CODE:

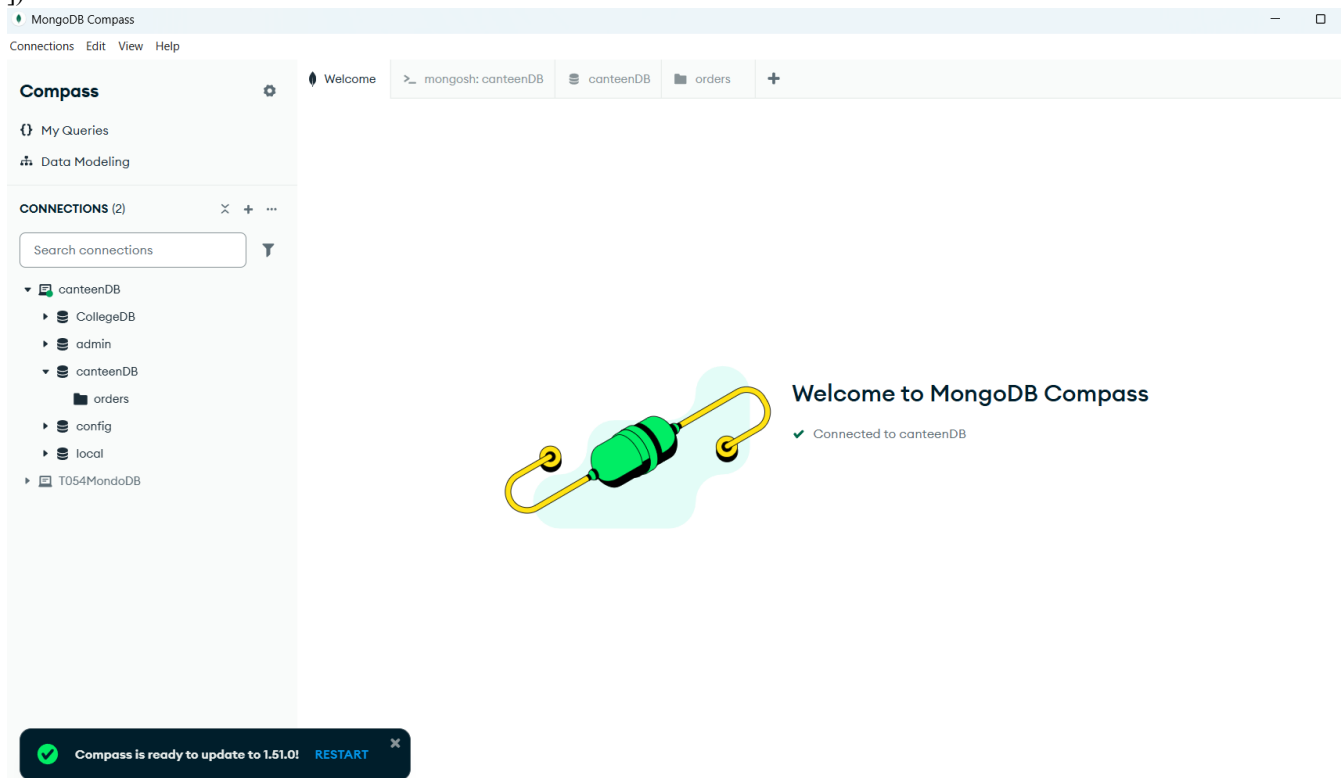
MongoDB Code:

```
use canteenDB
#insert single data
db.orders.insertOne({
  name: "Priti Yadav",
  rollNo: "54",
  course: "B.Sc. IT",
  food: "Vada Pav",
  quantity: 2,
  price: 20,
  total: 40
})
#find orders
db.orders.find()
#insert Many data
db.orders.insertMany([
  {
    name: "Rahul Sharma",
    rollNo: "55",
    course: "B.Sc. IT",
    food: "Samosa",
    quantity: 2,
    price: 15,
    total: 30
  },
```

```

{
  name: "Priya Patil",
  rollNo: "56",
  course: "B.Sc. IT",
  food: "Cold Coffee",
  quantity: 1,
  price: 60,
  total: 60
}
])

```



> _MONGOSH

```

> use("canteenDB")
< switched to db canteenDB
> db["orders"].find()
<
> db.orders.insertOne({
  name: "Priti Yadav",
  rollNo: "54",
  course: "B.Sc. IT",
  food: "Vada Pav",
  quantity: 2,
  price: 20,
  total: 40
})

```

```

< {
  acknowledged: true,
  insertedId: ObjectId('6ab7d5f8028de285abbd631f')
}

```

```

> db.orders.find()
< {
  _id: ObjectId('6ab7d5f8028de285abbd631f'),
  name: 'Priti Yadav',
  rollNo: '54',
  course: 'B.Sc. IT',
  food: 'Vada Pav',
  quantity: 2,
  price: 20,
  total: 40
}

```

```

< {
  acknowledged: true,
  insertedIds: {
    '0': ObjectId('6ab7d617028de285abbd6320'),
    '1': ObjectId('6ab7d617028de285abbd6321')
  }
}

```

```

> db.orders.insertMany([
  {
    name: "Rahul Sharma",
    rollNo: "55",
    course: "B.Sc. IT",
    food: "Samosa",
    quantity: 2,
    price: 15,
    total: 30
  },
  {
    name: "Priya Patil",
    rollNo: "56",
    course: "B.Sc. IT",
    food: "Cold Coffee",
    quantity: 1,
    price: 60,
    total: 60
  }
])

> db.orders.find()
< {
  _id: ObjectId('6ab7d5f8028de285abbd631f'),
  name: 'Priti Yadav',
  rollNo: '54',
  course: 'B.Sc. IT',
  food: 'Vada Pav',
  quantity: 2,
  price: 20,
  total: 40
}
{
  _id: ObjectId('6ab7d617028de285abbd6320'),
  name: 'Rahul Sharma',
  rollNo: '55',
  course: 'B.Sc. IT',
  food: 'Samosa',
  quantity: 2,
  price: 15,
  total: 30
}
{
  _id: ObjectId('6ab7d617028de285abbd6321'),
  name: 'Priya Patil',
  rollNo: '56',
  course: 'B.Sc. IT',
  food: 'Cold Coffee',
  quantity: 1,
  price: 60,
  total: 60
}

```

Food API: <http://localhost:3000/food>

Orders API: <http://localhost:3000/orders>

localhost:3000/orders

Pretty print ☒

```

[
  {
    "_id": "6ab7d5f8028de285abbd631f",
    "name": "Priti Yadav",
    "rollNo": "54",
    "course": "B.Sc. IT",
    "food": "Vada Pav",
    "quantity": 2,
    "price": 20,
    "total": 40
  },
  {
    "_id": "6ab7d617028de285abbd6320",
    "name": "Rahul Sharma",
    "rollNo": "55",
    "course": "B.Sc. IT",
    "food": "Samosa",
    "quantity": 2,
    "price": 15,
    "total": 30
  },
  {
    "_id": "6ab7d617028de285abbd6321",
    "name": "Priya Patil",
    "rollNo": "56",
    "course": "B.Sc. IT",
    "food": "Cold Coffee",
    "quantity": 1,
    "price": 60,
    "total": 60
  }
]

```

localhost:3000/food

Pretty print ☒

```

[
  {
    "id": 1,
    "name": "Vada Pav",
    "price": 20
  },
  {
    "id": 2,
    "name": "Samosa",
    "price": 15
  },
  {
    "id": 3,
    "name": "Veg Sandwich",
    "price": 40
  },
  {
    "id": 4,
    "name": "Masala Maggi",
    "price": 50
  },
  {
    "id": 5,
    "name": "Cold Coffee",
    "price": 60
  }
]

```

CODE:

Service.js:

```

const express = require("express");
const mongoose = require("mongoose");
const cors = require("cors");
const app = express();
const PORT = 3000;
// Middleware
app.use(cors());

```

```

app.use(express.json());
// MongoDB Connection
mongoose.connect("mongodb://127.0.0.1:27017/canteenDB")
  .then(() => {
    console.log("MongoDB Connected Successfully");
  })
  .catch((error) => {
    console.log("MongoDB Connection Failed");
    console.log(error);
  });
// Home Route
app.get("/", (req, res) => {
  res.send("College Canteen Pre-Order System is Running");
});
// GET /food
app.get("/food", (req, res) => {
  const foodItems = [
    { id: 1, name: "Vada Pav", price: 20 },
    { id: 2, name: "Samosa", price: 15 },
    { id: 3, name: "Veg Sandwich", price: 40 },
    { id: 4, name: "Masala Maggi", price: 50 },
    { id: 5, name: "Cold Coffee", price: 60 }
  ];
  res.json(foodItems);
});
// POST /orders
app.post("/orders", async (req, res) => {
  try {
    const order = req.body;
    await mongoose.connection.db
      .collection("orders")
      .insertOne(order);
    res.json({
      message: "Order placed successfully",
      order: order
    });
  } catch (error) {
    console.log(error);
    res.status(500).json({
      message: "Error placing order"
    });
  }
});
// GET /orders
app.get("/orders", async (req, res) => {
  try {
    const orders = await mongoose.connection.db
      .collection("orders")
      .find()
      .toArray();
    res.json(orders);
  } catch (error) {
    console.log(error);
    res.status(500).json({
      message: "Error fetching orders"
    });
  }
});
// Start Server
app.listen(PORT, () => {
  console.log(`Server is running on http://localhost:${PORT}`);
});

```

App.jsx:

```
import { useEffect, useState } from "react";
import axios from "axios";
function App() {
  const [foodItems, setFoodItems] = useState([]);
  const [orders, setOrders] = useState([]);
  const [name, setName] = useState("");
  const [rollNo, setRollNo] = useState("");
  const [course, setCourse] = useState("");
  const [food, setFood] = useState("");
  const [quantity, setQuantity] = useState(1);
  // Get Food Items
  const getFood = () => {
    axios
      .get("http://localhost:3000/food")
      .then((response) => {
        setFoodItems(response.data);
      })
      .catch((error) => {
        console.log(error);
      });
  };
  // Get Orders
  const getOrders = () => {
    axios
      .get("http://localhost:3000/orders")
      .then((response) => {
        setOrders(response.data);
      })
      .catch((error) => {
        console.log(error);
      });
  };
  // Load Food and Orders
  useEffect(() => {
    getFood();
    getOrders();
  }, []);
  // Place Order
  const placeOrder = () => {
    if (
      name === "" ||
      rollNo === "" ||
      course === "" ||
      food === ""
    ) {
      alert("Please fill all details");
      return;
    }
    const selectedFood = foodItems.find(
      (item) => item.name === food
    );
    const newOrder = {
      name: name,
      rollNo: rollNo,
      course: course,
      food: selectedFood.name,
      quantity: Number(quantity),
      price: selectedFood.price,
      total: selectedFood.price * Number(quantity)
    };
    axios
```

```

.post("http://localhost:3000/orders", newOrder)
.then((response) => {
  alert(response.data.message);
  setName("");
  setRollNo("");
  setCourse("");
  setFood("");
  setQuantity(1);
  getOrders();
})
.catch((error) => {
  console.log(error);
});
});
return (
  <div>
    <h1>College Canteen Pre-Order System</h1>
    <hr />
    { /* Food Items */ }
    <h2>Available Food Items</h2>
    <table border="1" cellPadding="10">
      <thead>
        <tr>
          <th>Food Item</th>
          <th>Price</th>
        </tr>
      </thead>
      <tbody>
        {foodItems.map((item) => (
          <tr key={item.id}>
            <td>{item.name}</td>
            <td>₹ {item.price}</td>
          </tr>
        ))}
      </tbody>
    </table>
    <hr />
    { /* Student Order Form */ }
    <h2>Student Order Form</h2>
    <input
      type="text"
      placeholder="Student Name"
      value={name}
      onChange={(e) => setName(e.target.value)}
    />
    <br />
    <br />
    <input
      type="text"
      placeholder="Roll Number"
      value={rollNo}
      onChange={(e) => setRollNo(e.target.value)}
    />
    <br />
    <br />
    <input
      type="text"
      placeholder="Course"
      value={course}
      onChange={(e) => setCourse(e.target.value)}
    />
    <br />

```

```

    <br />
    <select
      value={food}
      onChange={(e) => setFood(e.target.value)}
    >
      <option value="">
        Select Food Item
      </option>
      {foodItems.map((item) => (
        <option
          key={item.id}
          value={item.name}
        >
          {item.name} - ₹ {item.price}
        </option>
      ))}
    </select>
    <br />
    <br />
    <input
      type="number"
      min="1"
      value={quantity}
      onChange={(e) => setQuantity(e.target.value)}
    />
    <br />
    <br />
    <button onClick={placeOrder}>
      Place Order
    </button>
    <hr />
    { /* Orders */ }
    <h2>Placed Orders</h2>
    <table border="1" cellPadding="10">
      <thead>
        <tr>
          <th>Name</th>
          <th>Roll No</th>
          <th>Course</th>
          <th>Food</th>
          <th>Quantity</th>
          <th>Price</th>
          <th>Total</th>
        </tr>
      </thead>
      <tbody>
        {orders.map((order) => (
          <tr key={order._id}>
            <td>{order.name}</td>
            <td>{order.rollNo}</td>
            <td>{order.course}</td>
            <td>{order.food}</td>
            <td>{order.quantity}</td>
            <td>₹ {order.price}</td>
            <td>₹ {order.total}</td>
          </tr>
        ))}
      </tbody>
    </table>
  </div>
);}
export default App;

```

OUTPUT:

```
PS C:\Users\priti yadav\OneDrive\Desktop\Mern Practical> cd Exprac
PS C:\Users\priti yadav\OneDrive\Desktop\Mern Practical\Exprac> cd my-react
PS C:\Users\priti yadav\OneDrive\Desktop\Mern Practical\Exprac\my-react> npm install express mongoose cors

up to date, audited 248 packages in 12s

63 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\priti yadav\OneDrive\Desktop\Mern Practical\Exprac\my-react> npm install axios

up to date, audited 248 packages in 2s

63 packages are looking for funding
  run `npm fund` for details

found 0 vulnerabilities
PS C:\Users\priti yadav\OneDrive\Desktop\Mern Practical\Exprac\my-react> npm run dev

> my-react@0.0.0 dev
> vite

VITE v8.3.0 ready in 1067 ms

→ Local:   http://localhost:5173/
→ Network: use --host to expose
→ press h + enter to show help

o
o
```

```
PS C:\Users\priti yadav\OneDrive\Desktop\Mern Practical\Exprac\my-react> cd..
PS C:\Users\priti yadav\OneDrive\Desktop\Mern Practical\Exprac> node service.js
Server is running on http://localhost:3000
MongoDB Connected Successfully
[]
```

← localhost:5173

College Canteen Pre-Order System

Available Food Items

Food Item	Price
Vada Pav	₹20
Samosa	₹15
Veg Sandwich	₹40
Masala Maggi	₹50
Cold Coffee	₹60

Student Order Form

Placed Orders

Name	Roll No	Course	Food	Quantity	Price	Total
Priti Yadav	54	B.Sc. IT	Vada Pav	2	₹20	₹40
Rahul Sharma	55	B.Sc. IT	Samosa	2	₹15	₹30
Priya Patil	56	B.Sc. IT	Cold Coffee	1	₹60	₹60

College Canteen Pre-Order System

Available Food Items

Food Item	Price
Vada Pav	₹20
Samosa	₹15
Veg Sandwich	₹40
Masala Maggi	₹50
Cold Coffee	₹60

Student Order Form

Placed Orders

Name	Roll No	Course	Food	Quantity	Price	Total
Priti Yadav	54	B.Sc. IT	Vada Pav	2	₹20	₹40
Rahul Sharma	55	B.Sc. IT	Samosa	2	₹15	₹30
Priya Patil	56	B.Sc. IT	Cold Coffee	1	₹60	₹60

College Canteen Pre-Order System

localhost:5173 says

Order placed successfully

OK

Food Item	Price
Vada Pav	₹20
Samosa	₹15
Veg Sandwich	₹40
Masala Maggi	₹50
Cold Coffee	₹60

Student Order Form

College Canteen Pre-Order System

Available Food Items

Food Item	Price
Vada Pav	₹20
Samosa	₹15
Veg Sandwich	₹40
Masala Maggi	₹50
Cold Coffee	₹60

Student Order Form

Placed Orders

Name	Roll No	Course	Food	Quantity	Price	Total
Priti Yadav	54	B.Sc. IT	Vada Pav	2	₹20	₹40
Rahul Sharma	55	B.Sc. IT	Samosa	2	₹15	₹30
Priya Patil	56	B.Sc. IT	Cold Coffee	1	₹60	₹60
keshav jha	22	B.COM	Masala Maggi	1	₹50	₹50

pretty print ☒

```
[
  {
    "_id": "6ab7d5f8028de285abbd631f",
    "name": "Priti Yadav",
    "rollNo": "54",
    "course": "B.Sc. IT",
    "food": "Vada Pav",
    "quantity": 2,
    "price": 20,
    "total": 40
  },
  {
    "_id": "6ab7d617028de285abbd6320",
    "name": "Rahul Sharma",
    "rollNo": "55",
    "course": "B.Sc. IT",
    "food": "Samosa",
    "quantity": 2,
    "price": 15,
    "total": 30
  },
  {
    "_id": "6ab7d617028de285abbd6321",
    "name": "Priya Patil",
    "rollNo": "56",
    "course": "B.Sc. IT",
    "food": "Cold Coffee",
    "quantity": 1,
    "price": 60,
    "total": 60
  },
  {
    "_id": "6ab7db6bef134b5eb6e935d7",
    "name": "keshav jha",
    "rollNo": "22",
    "course": "B.COM",
    "food": "Masala Maggi",
    "quantity": 1,
    "price": 50,
    "total": 50
  }
]
```

Advantages

1. Easy Pre-Ordering: Students can order food before reaching the canteen.
2. Saves Time: Reduces waiting time and long queues at the canteen.
3. Simple Interface: Students can easily view food items, prices, and place orders.
4. Automatic Calculation: The total price is calculated automatically based on quantity.
5. Data Storage: Order details are stored securely in MongoDB.
6. Easy Data Retrieval: Placed orders can be retrieved using the GET API.
7. Real-Time Communication: React communicates with the Node.js/Express backend using Axios.
8. Reduced Manual Work: The system reduces manual order-taking and record keeping.
9. Scalable System: More food items and features can be added in the future.
10. Uses Modern Technologies: The project demonstrates React, Axios, Node.js, Express.js, and MongoDB together.

Conclusion

The College Canteen Pre-Order System successfully provides a simple and efficient way for students to view food items, enter their details, select food and quantity, and place orders. The orders are sent through a POST API and stored in MongoDB, while the GET API retrieves and displays the placed orders. The project demonstrates the integration of React, Axios, Node.js, Express.js, and MongoDB to develop a basic full-stack application.